Interim Report

Templating in Standard ML

Ja Ying Lew

Supervisor: Gergely Buday

Department of Biological, Chemical and Materials Engineering

University of Sheffield

November 24, 2025

Contents

1. Background to Project

Most template systems separate markup and tags from application logic, particularly Mustache highlights this through its "logic-less" design by prohibiting embedding traditional conditional statements or loops within templates [1]. The strict separation of all logic residing in the view model and templates being able to format specifications has led to Mustache's widespread adoption across numerous programming languages. [2] Despite having a broad implementation and being able to provide readable output, loopholes arise when they require preprocessing/ massaging of the data to produce a 'logical' output [3, 4]. Exploiting standard ML's strong static type system, strong parsing and pattern matching abilities, these characteristics may guarantee template correctness and type-safe data processing[5, 6]. This project investigates a Mustache template engine in Standard ML, investigating two processing strategies: (1) preprocessing, where Standard ML transforms JSON data before template application, and [3] (2) postprocessing, where Standard ML manipulates Mustache-generated output. Both approaches leverage Standard ML's strong static type system and pattern matching to address the data massaging challenges of logic-less templates. The implementation parses Mustache syntax, processes data contexts, and generates output while maintaining type safety throughout the pipeline. Beyond creating a functional tool, the project evaluates Standard ML's suitability for template engine implementation in both preprocessing and postprocessing roles, examining advantages (type safety, pattern matching, parsing) and limitations (library ecosystem, tooling) encountered during development.

Listing 1.1: Mustache template example from OpenAPI Generator

```
''' csharp
using {{packageName}}.{{modelPackage}};
using System;
{{^composedSchemas.oneOf}}
{{#vars}}{{#isDecimal}}{{^required}}{{dataType}}? {{nameInCamelCase}} = "
    example {{nameInCamelCase}}"; {{/required}} {{#required}}{{dataType}} {{
    nameInCamelCase}} = "example {{nameInCamelCase}}";
{{/required}}{{/isDecimal}}{{#isNumber}}{{^required}}{{dataType?}} {{
    nameInCamelCase}} = "example {{nameInCamelCase}}";
{{/required}}{{#required}}{{dataType}}? {{nameInCamelCase}} = "{{
```

```
    nameInCamelCase}}" ; {{/required}}{{/isNumber}}{{#isBoolean}}{{^required}}
{{dataType}} {{nameInCamelCase}} = //"True"; {{/required}} {{#required}}
{{dataType}} {{nameInCamelCase}} = //"True"; {{/required}}{{/isBoolean}}{{#
    isString}}
{{^required}}
{{{dataType}}} {{nameInCamelCase}} = "example {{nameInCamelCase}}"; {{/
    required}}{{dataType}} {{nameInCamelCase}} = "{{nameInCamelCase}}" ;{{/
    required}}{{/isString}}{{#isModel}}{{^required}}
{{dataType}} {{nameInCamelCase}} = new {{{dataType}}}(); {{/required}}
{{#required}}
{{{dataType}}} {{nameInCamelCase}} = new {{{dataType}}}(); {{/required}}{{/
    isModel}}{{#isContainer}}{{^required}}
{{dataType}} {{nameInCamelCase}} = new {{{dataType}}}(); {{/required}} {{#
    required}}
{{dataType}} {{nameInCamelCase}} = new {{{dataType}}}(); {{/required}}{{/
    isContainer}}{{/vars}}

{{classname}} {{#lamba.camelcase}} {{classname}} {{/lamba.camelcase}}Instance
    = new {{classname}}(
    {{#vars}}{{nameInCamelCase}}: {{nameInCamelCase}}{{^-last}},
    {{/-last}}{{/vars}});
    {{/composedSchemas.oneOf}}

'''
```

1.0.1. Generated API Documentation Example

The Mustache template generates comprehensive API documentation including property tables and usage examples. Table 1.1 shows the generated property documentation for the `A2BBreakdown` model.

Table 1.1: Generated API properties for A2BBreakdown model

Name	Type	Description	Notes
Total	decimal	The total value of all components within this category	optional
Currency	string	The currency. Applies to the Total and all components	optional
Components	Dictionary¡string, decimal¿	Individual components that make up the category	optional

The template also generates usage examples in C#:

Listing 1.2: Generated C# usage example

```
using Lusid.Sdk.Model;
using System;

decimal? total = "example total";
string currency = "example currency";
Dictionary<string, decimal> components = new Dictionary<string, decimal>();

A2BBreakdown a2BBreakdownInstance = new A2BBreakdown(
    total: total,
    currency: currency,
    components: components);
```

**not sure if I should include this since there is no articles that explictly says this, but *From practical experience, Mustache templates require manual completion of commented placeholders (e.g., //"True"), necessitating additional verification steps. This manual intervention increases both error risk and development time.*

2. Introduction: Standard ML and Mustache

Robin Milner developed ML while working on the LCF theorem prover, initially implemented in Lisp at Stanford. Although Lisp provided useful primitives such as lists and exception handling (catch/throw), it lacked proper function closures, used dynamic typing, and had no support for abstract types. Milner addressed these limitations in ML by introducing static type checking to catch errors at compile time, abstract types to encapsulate data structures, and proper function closures—features that proved essential for building reliable automated theorem provers [7]. The language was formally standardised in 1990 and revised in 1997 as Standard ML '97, becoming one of the few programming languages with a fully formal definition at the time. [8].

While Standard ML has not achieved the widespread adoption of languages like Python, it remains actively used in specialized domains. Its continued relevance stems from its safety guarantees—all errors that could derail an ML program are detected at compile-time or handled at run-time—and its formal specification, making it particularly appealing for research and industrial-strength applications requiring high reliability [8]. Standard ML's primary applications include compiler construction, programming language research, theorem proving, bioinformatics, and financial systems: one application of this is at IT University of Copenhagen's enterprise architecture, which comprises approximately 100,000 lines of SML code [8, 9].

The language offers several distinctive advantages for parsing and language processing tasks. Its Hindley-Milner type system provides automatic type inference without explicit annotations while ensuring type safety through formal guarantees that well-typed programs cannot cause runtime type errors [9]. Standard ML's strong support for algebraic datatypes, combined with comprehensive pattern matching and pattern-exhaustiveness checking, makes it particularly well-suited for representing and manipulating abstract syntax trees [8]. As VandenBerghe notes, ML's type constructors implement tagged unions that are "wonderful for describing ASTs," with pattern matching making source code "incredibly readable" for functions operating on complex data structures [6]. The language also features automatic garbage collection that performs comparably to C++ manual memory management, tail recursion optimization for efficient tree traversals, and

a sophisticated module system for organizing larger programs [6].

However, Standard ML has limitations that affect its adoption. The language ecosystem is significantly smaller than mainstream alternatives, with fewer third-party libraries and limited GUI support [6]. Its syntax and functional programming paradigm present a steeper learning curve for developers accustomed to imperative languages. Additionally, Standard ML is unsuitable for certain problem domains such as embedded systems programming or real-time DSP applications [6].

Mustache is a logic-less template system developed by Chris Wanstrath and first released in October 2009, inspired by Google's ctemplate library [10]. Emerging during GitHub's early growth, Mustache addressed the challenge of generating dynamic web content while maintaining strict separation between presentation and application logic in Ruby on Rails applications [10]. The system's fundamental design principle prohibits explicit control flow statements—no if-else conditionals or for loops—relying instead on sections and inverted sections to handle conditional rendering and iteration through data structures. However, this is disputed as it still has control statements implictly as it uses 'section tags processing lists and anonymous functions (lambdas)' [10].

Mustache has achieved remarkable adoption, with implementations available in over 40 programming languages including ActionScript, C++, Clojure, Java, JavaScript, Python, Ruby, Rust, Haskell, and OCaml [2]. Its extension, Handlebars.js, demonstrates the template system's influence with over 5.3 million monthly downloads on NPM, surpassing even popular frameworks like Angular.js and React in distribution metrics [10]. Current applications span mobile and web development, configuration file generation, source code generation, build automation tools, and API client generation through systems like OpenAPI Generator [10, 11].

What distinguishes Mustache is its strict enforcement of the logic-less paradigm through a language-agnostic specification (version 1.3.0) that ensures consistent behavior across all implementations [10]. Templates use double curly braces ({{`variable`}}) for variable substitution, section tags ({`#section`}} ... {`/section`}}) for iteration and conditionals, inverted sections ({`^section`}}) for negative conditions, and partials ({ `partial`}}) for template composition. This design makes templates portable across programming environments and readable by non-programmers, while HTML escaping is applied by default to prevent cross-site scripting attacks.

Despite these advantages, the logic-less approach introduces significant challenges. Critics argue that prohibiting logic in templates necessitates bloated view models with comprehensive getter methods for data access, and forces extensive data preprocessing or "massaging" before templates can render properly [3, 4]. Boronine contends that this supposedly clean separation actually violates separation of concerns by pushing pre-

sentational logic into business code [4]. As templates grow in size and complexity, they become increasingly difficult to manage without the flexibility of conditional logic [3]. Every variation to the view requires updating both the view model and template, increasing maintenance burden.

2.0.1. Aims and Objectives

This project implements a Mustache template engine in Standard ML, investigating two processing strategies: preprocessing (transforming JSON data before template application) and postprocessing (manipulating Mustache-generated output). Leveraging Standard ML's strong static type system, pattern matching, and algebraic datatypes, the implementation addresses data preprocessing challenges inherent to logic-less templates [3, 4] while providing compile-time guarantees about template correctness. Beyond creating a functional tool, the project evaluates Standard ML's suitability for template engine implementation, examining advantages (type safety, pattern matching, parsing) and limitations (library ecosystem, syntax learning curve [6], tooling) encountered during development.

The project spans two semesters. Semester one focuses on learning Standard ML fundamentals (type system, pattern matching, functional paradigm), conducting comprehensive literature review on template systems and Standard ML's language processing applications, producing the Survey and Analysis document, and beginning prototype implementation exploring Mustache syntax parsing and data structure representation. Semester two emphasises full template engine implementation with both processing approaches, integration and comprehensive testing, refinement through comparative analysis against existing Mustache implementations, and six weeks dedicated to dissertation write-up.

3. Literature Review

3.1. Introduction

This literature review aims to examine alternative template systems in software engineering. Template systems address the challenge and the importance of separating presentation logic from application code, a principle known as separation of concerns [1].

3.2. Template Systems: Purpose and Design

Template systems separate presentation markup from application logic, addressing maintenance difficulties and security vulnerabilities that arise when code intermingles with markup [1]. By providing specialised syntax for presentation structures that reference application-provided data, template engines enforce architectural boundaries between view and model layers.

Effective template system design balances expressiveness against simplicity. While powerful template languages enable complex logic directly in templates, they risk duplicating business logic and violating separation of concerns [1]. Conversely, overly restrictive systems force bloated view models with excessive getter methods and require substantial data preprocessing before rendering [3]. A well-designed template system should enforce separation without imposing excessive preprocessing burden or pushing presentational logic into business code [3, 4]. Different engines resolve these tensions through varying philosophies about appropriate boundaries between presentation and logic.

Template system design involves tension between expressiveness and simplicity. Powerful template languages allow complex logic directly in templates, potentially duplicating business logic and violating separation of concerns. Conversely, restrictive systems may force awkward workarounds when presentation requires dynamic behavior [12]. Different engines resolve this tension differently, reflecting varying philosophies about boundaries between presentation and logic.

3.2.1. Alternative Templating Systems

Modern HTML Templating Engines:

- **Handlebars** extends Mustache by adding helpers, block expressions, and partials while maintaining logic-less philosophy, compiling templates into JavaScript functions for faster execution [13, 14]. It provides built-in helpers such as `if`, `each`, and `unless`, and supports precompilation for improved runtime performance.

- **Nunjucks**, inspired by Python's Jinja2, provides block inheritance, autoescaping, macros, and asynchronous control, making it significantly more feature-rich than Mustache while maintaining readable syntax [15]. It offers template inheritance with block overriding and automatic HTML escaping for XSS prevention.

Logic-Friendly Templating Languages:

- **EJS (Embedded JavaScript)** allows developers to write plain JavaScript directly in templates using special delimiters, with `<% %>` for code execution and `<%= %>` for output, eliminating the need to learn new syntax [16]. It caches intermediate functions for fast execution and provides full JavaScript expressiveness with server-side rendering support.

- **Jinja2** is Python's fast and extensible templating engine that supports template inheritance, macros, HTML autoescaping, and sandboxed environments for safely rendering untrusted templates [4, 17]. It offers more flexibility than Django's templating system and features a powerful extension system with custom filters and tests.

Language-Integrated Templating:

- **React/JSX** integrates markup directly into JavaScript as first-class code through components, where JSX expressions compile to regular JavaScript function calls that can be used in conditionals, loops, and variable assignments [18]. It provides a component-based architecture with full JavaScript expressiveness and seamless TypeScript integration for type safety.

- **Svelte** shifts template processing from browser runtime to compile-time, generating highly optimized vanilla JavaScript without framework overhead, producing smaller bundles and better performance than runtime frameworks [19]. It eliminates runtime overhead through compile-time processing and provides a reactive programming model with minimal boilerplate code.

These alternatives represent different templating philosophies: Handlebars and Nunjucks offer richer syntax while remaining declarative; EJS and Jinja2 provide full programming language power with security considerations; React/JSX and Svelte integrate

templates as native code. The choice depends on logic complexity requirements, security concerns, language ecosystem, performance priorities, and whether compile-time or runtime processing is preferred.

3.3. Functional Languages for Template Processing

Functional programming languages bring distinctive characteristics to template systems: strong static typing that catches errors at compile time, pattern matching for elegant data structure manipulation, and algebraic datatypes for representing abstract syntax trees [6, 9]. Standard ML, OCaml, and Haskell share common ancestry in the ML tradition but differ significantly in design philosophy and practical characteristics.

Standard ML emphasises formal rigor, being one of few programming languages with a complete formal definition [7, 8]. Its strict (call-by-value) evaluation strategy makes behavior predictable, and its pattern matching with algebraic datatypes excels at representing abstract syntax trees [5]. However, it suffers from a limited library ecosystem, requiring custom JSON parsing and file I/O implementations [6].

OCaml prioritises pragmatism and industrial applicability, offering superior performance through aggressive compiler optimizations and a richer standard library via OPAM [9]. Jingoo provides Jinja2-compatible templating for OCaml applications [20]. OCaml relaxes Standard ML's theoretical purity for practical flexibility, adding object-oriented features and first-class modules.

Haskell represents 'lazy' functional programming, evaluating expressions only when needed [9]. Its advanced type system enables sophisticated compile-time template solutions: Hamlet uses Template Haskell metaprogramming to compile templates with type-checked interpolations and automatic XSS prevention [21, 22], while Blaze-html represents HTML as composable functions for maximum type safety [23].

This project selects Standard ML for several reasons: its formal definition and proven compiler construction track record suggest suitability for parsing tasks [5]; strict evaluation simplifies reasoning about template rendering compared to Haskell's laziness; pattern matching naturally represents Mustache's syntax; and crucially, no Standard ML Mustache implementation exists, representing a genuine research gap [2]. While Standard ML's limited ecosystem and declining adoption present challenges [6], these tradeoffs accept reduced convenience for theoretical clarity and evaluation opportunities.

4. Current Status of Work

Work to date has focused on two parallel tracks: learning Standard ML and conducting literature review for this interim report.

4.0.1. Standard ML Proficiency

Standard ML familiarisation has proceeded through the Gilmore exercise sets, with particular emphasis on Chapters 9 and 10 covering the module system (signatures, structures, and functors). This establishes the foundation needed for implementing the template engine architecture using Standard ML's module facilities.

4.0.2. Literature Review

The interim report has been developed systematically: general review of template system design principles; analysis of Mustache's logic-less philosophy and documented limitations; investigation of Standard ML's history, formal definition, and compiler construction applications; and examination of alternative template engines (Handlebars, Nunjucks, EJS, Jinja2, React/JSX, Svelte) to establish context.

4.0.3. Planned Work

Following interim submission, implementation will begin with prototype parsing of Mustache syntax using algebraic datatypes and pattern matching, exploring both preprocessing and postprocessing strategies.

5. Self-Review

Coming from a Biomedical Engineering background, I initially underestimated the foundational knowledge gap between disciplines. The Computer Science terminology alone—terms that would be second nature to CS students—presented a significant barrier. Early progress was slower than expected as I encountered concepts like signatures, primary and secondary compuation, and standard ML, each requiring substantial background reading to understand properly.

Rather than attempting to rush through unfamiliar material, I adopted a deliberate, bottom-up approach: thoroughly understanding each new term and concept before moving forward. While frustrating at times, this strategy has proven worthwhile. Individual pieces that initially seemed disconnected are now forming a coherent picture, and I am beginning to appreciate the elegance of functional programming's approach to problems I had previously only encountered in imperative contexts

6. Project Management

Write your project management plan here.

Programme of Work

Immediate Tasks (Interim Report Refinement)

- Clarify distinction between implementation language (Standard ML) and target language (Python) for the template system
- Add reference to CM-Lex and CM-Yacc tool to demonstrate Standard ML's parsing capabilities
- Include proper reference for Mustache's limitation and emphasise this research gap in the introduction
- Add detailed descriptions of alternative template engines (Nunjucks, Handlebars, Jinja2, EJS, React/JSX, Svelte, Django Templates) with 2-3 sentences each
- Reference Appel's *Modern Compiler Implementation in ML* to support Standard ML's suitability for parsing tasks
- Finalise Harvard-style referencing system and verify compliance with institutional citation guidelines

Implementation Phase (Post-Interim Submission)

- Design and implement parser for Mustache syntax using Standard ML algebraic datatypes and pattern matching
- Evaluate CM-Lex and CM-Yacc as potential parsing tools for the project
- Develop prototype implementation exploring both preprocessing and postprocessing strategies
- Implement core Mustache features (variables, sections, inverted sections, comments) targeting Python output

- Design and test template engine architecture using Standard ML's module system (signatures, structures, functors)
- Conduct comparative evaluation against existing Mustache implementations, measuring correctness beyond the documented 70% threshold

Evaluation and Documentation

- Perform honest evaluation of whether Standard ML proved suitable for template system implementation
- Document any challenges or limitations encountered with the Standard ML toolchain
- Assess whether the implementation outperforms existing Mustache implementations
- Prepare final dissertation incorporating implementation results and critical analysis

A. Project Timeline

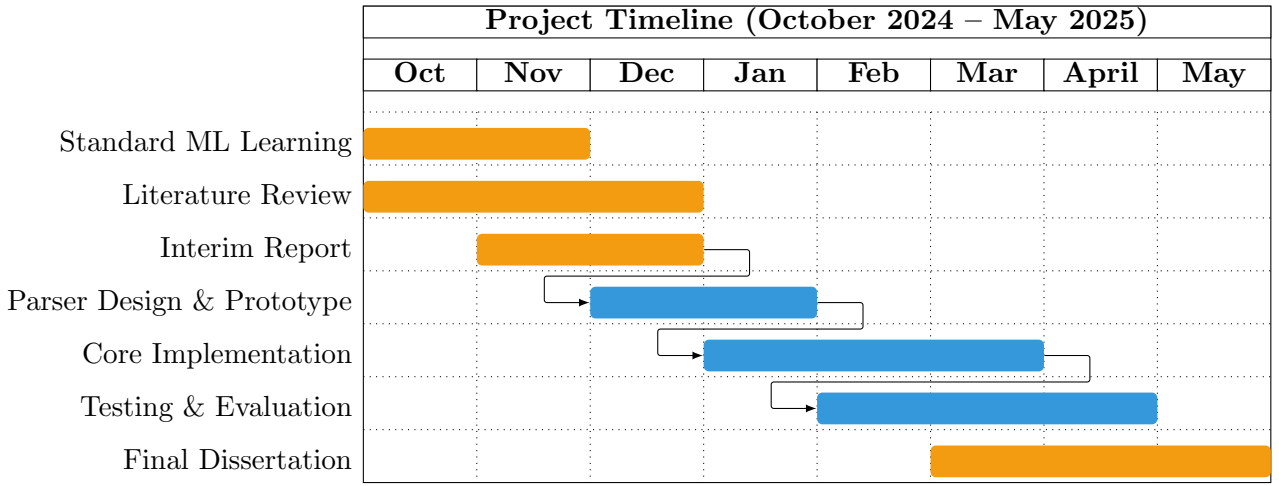

Figure A.1: Project Gantt chart showing key phases and milestones from October 2024 to May 2025

Bibliography

[1] T. J. Parr, "Enforcing strict model-view separation in template engines," in *Proceedings of the 13th International Conference on World Wide Web*, ser. WWW '04. New York, NY, USA: ACM, May 2004, pp. 224–233.

[2] "Mustache - logic-less templates," Official Website. [Online]. Available: https://mustache.github.io/

[3] S. Pottavathini. (2012, Oct.) The case against logic-less templates. eBay Inc. [Online]. Available: https://innovation.ebayinc.com/stories/the-case-against-logic-less-templates/

[4] A. Boronine. (2012, Sep.) Cult of logic-less templates. Alexei Boronine's Blog. [Online]. Available: https://www.boronine.com/2012/09/07/Cult-Of-Logic-less-Templates/

[5] A. W. Appel and D. B. MacQueen, "A standard ml compiler," *Technical Report, Department of Computer Science, Princeton University*, 1997, accessed: 2025-11-21. [Online]. Available: https://www.cs.princeton.edu/~appel/papers/97.pdf

[6] Y. U. Flint, "The case for ml," n.d., accessed: 2025-11-21. [Online]. Available: https://flint.cs.yale.edu/cs421/case-for-ml.html

[7] D. MacQueen, R. Harper, and J. Reppy, "The history of standard ml," *Proceedings of the ACM on Programming Languages*, vol. 4, no. HOPL, 2020. [Online]. Available: https://dl.acm.org/doi/abs/10.1145/3386336

[8] "Standard ml," Wikipedia, accessed: 2025-11-21. [Online]. Available: https://en.wikipedia.org/wiki/Standard_ML

[9] "Ml (programming language)," Wikipedia, accessed: 2025-11-21. [Online]. Available: https://en.wikipedia.org/wiki/ML_(programming_language)

[10] "Mustache (template system)," Wikipedia, accessed: 2025-11-21. [Online]. Available: https://en.wikipedia.org/wiki/Mustache_(template_system)

[11] *Mustache Manual*, Mustache. [Online]. Available: https://mustache.github.io/mustache.5.html

[12] "What's the advantage of logic-less template such as mustache?" Stack Overflow. [Online]. Available: https://stackoverflow.com/questions/3896730/whats-the-advantage-of-logic-less-template-such-as-mustache

[13] J-Gallo. (2016, Feb.) Mustache vs Handlebars. [Online]. Available: https://medium.com/@JuaniGallo/mustache-vs-handlebars-42e531eca252

[14] H. Team, "Handlebars," Official Website, accessed: 2025-11-21. [Online]. Available: https://handlebarsjs.com/

[15] Mozilla, "Nunjucks: A powerful templating engine with inheritance," GitHub, accessed: 2025-11-21. [Online]. Available: https://mozilla.github.io/nunjucks/

[16] E. Team, "Ejs – embedded javascript templates," Official Website, accessed: 2025-11-21. [Online]. Available: https://ejs.co/

[17] Pallets, "Jinja," Official Website, accessed: 2025-11-21. [Online]. Available: https://palletsprojects.com/p/jinja/

[18] R. Team, "Writing markup with jsx," React Documentation, accessed: 2025-11-21. [Online]. Available: https://react.dev/learn/writing-markup-with-jsx

[19] S. Team, "Svelte documentation," Official Website, accessed: 2025-11-21. [Online]. Available: https://svelte.dev/docs

[20] T. Bunko, "Jingoo: Template engine for ocaml," GitHub. [Online]. Available: https://github.com/tategakibunko/jingoo

[21] "Shakespearean templates," Yesod Book. [Online]. Available: https://www.yesodweb.com/book/shakespearean-templates

[22] M. Snoyman, *Yesod Web Framework Book*. Yesod. [Online]. Available: https://www.yesodweb.com/book

[23] J. Van Der Jeugt, "Blazehtml: A blazingly fast html combinator library," Personal Website. [Online]. Available: https://jaspervdj.be/blaze/